HDOC DAY-59

Write a C program that reads a string s. Pass the string to a function named longest_unique_substring() using its address. Use pointers to pass the string between the calling function and longest_unique_substring(). The function longest_unique_substring() must:

- Find the longest substring of s that contains no repeated characters.
- Calculate and display the length of the longest substring.
- Treat uppercase and lowercase letters as different characters.
- Display 0 if the input string is empty.

Example 1:

String: abcabcbb

Output:

Length of longest substring: 3

[Explanation: The longest substrings without repeating characters include: abc. Their length is 3.]

Example 2:

String: bbbbb

Output:

Length of longest substring: 1

[Explanation: The longest substring without repeating characters is: b]

Example 3:

String: pwwkew

Output:

Length of longest substring: 3

[Explanation: The longest substrings without repeating characters include: wke. Their length is 3.]

Example 4:

String: ` `

Output: 0

Algorithm

1. Read the string s from the user.
2. Pass its address to longest_unique_substring() using the pointer char *s.
3. Create an array last[256] to store the last position of each character.
4. Initialize all positions in last to -1.
5. Traverse the string character by character.
6. If the current character has already occurred in the current substring, move start to the position after its previous occurrence.
7. Calculate the current substring length as i - start + 1.
8. Update maxLength whenever a longer substring is found.
9. Display maxLength.
10. If the input string is empty, the loop does not execute and the program displays 0.

- # C Program:

```
#include <stdio.h>
#include <string.h>

void longest_unique_substring(char *s)
{
    int last[256];
    int start = 0, maxLength = 0;
    int i;

    for (i = 0; i < 256; i++)
        last[i] = -1;

    for (i = 0; s[i] != '\0'; i++)
    {
        unsigned char ch = s[i];
```

```
        if (last[ch] >= start)
            start = last[ch] + 1;

        last[ch] = i;

        if (i - start + 1 > maxLength)
            maxLength = i - start + 1;
    }

    printf("Length of longest substring: %d\n", maxLength);
}

int main()
{
    char s[1000];

    printf("Enter string: ");
    fgets(s, sizeof(s), stdin);

    /* Remove newline added by fgets */
    s[strcspn(s, "\n")] = '\0';

    /* Pass the address of the string */
    longest_unique_substring(s);

    return 0;
}
```

Test Case Table:

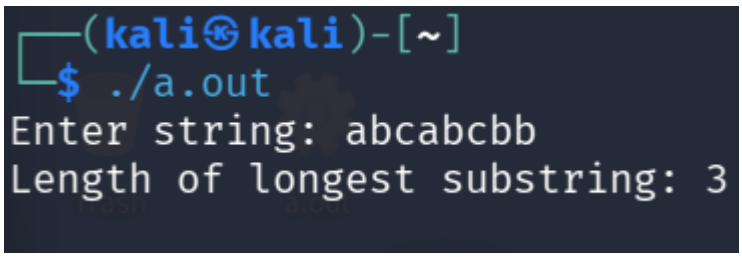
Test Case	Input	Expected Output
1	Abcabcbb	Length of longest substring: 3
2	Bbbbb	Length of longest substring: 1
3	Pwwkew	Length of longest substring: 3
4	Empty String	Length of longest substring:8

5	Abcdef	Length of longest substring: 6
6	aAbBcC	Length of longest substring: 6
7	AaA	Length of longest substring: 2

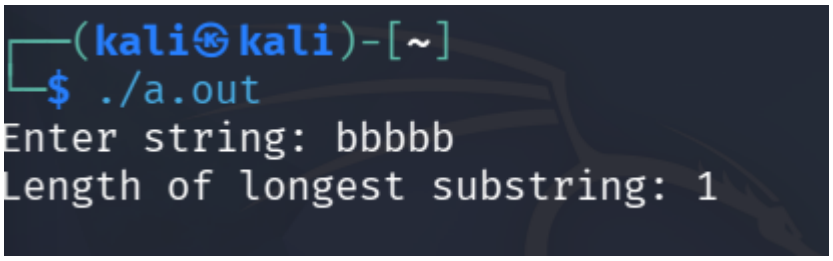
Compilation and Execution:

The following program has been compiled and executed successfully.

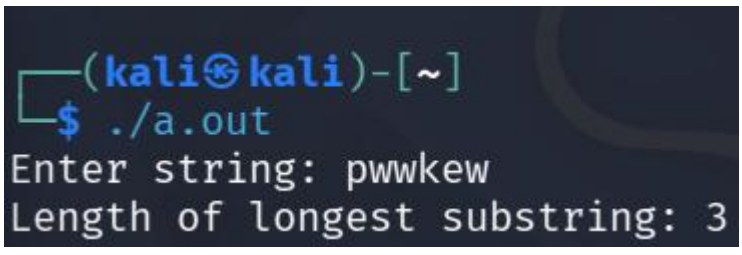
Test Case 1:



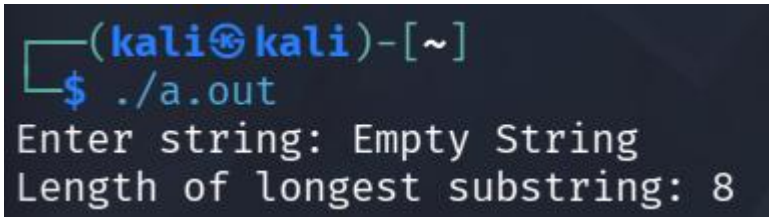
Test Case 2:



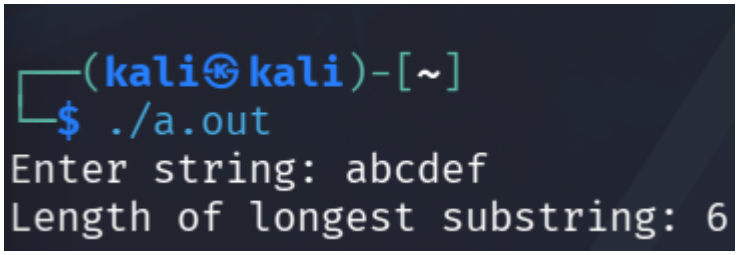
Test Case 3:



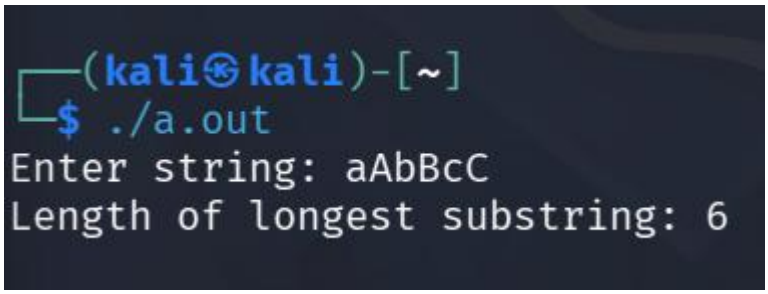
Test Case 4:



Test Case 5:



Test Case 6:



Test Case 7:

